Report File

On

Movie Recommendation System

Bachelor Of Technology

In

Computer Science and Engineering

Submitted to:
Ms. Farha Fatma
Assistant Professor
(Big Data Analytics)

Submitted by:
Harshita Hoiyani 1000018489
Ansh Mittal 1000018268
Shashank Bisht 1000018182
Zoha Fatima 1000017637

SCHOOL OF COMPUTING
DIT UNIVERSITY, DEHRADUN
(State Private University through State Legislature Act No. 10 of 2013 of Uttarakhand and approved by UGC)
Mussoorie Diversion Road, Dehradun, Uttarakhand - 248009, India.

PREFACE

- In the modern era of digital entertainment, recommendation systems have become an integral part of online streaming and e-commerce platforms. Services such as **Netflix, Amazon Prime, Spotify, and YouTube** rely heavily on recommendation systems to suggest personalized content to users, improving user engagement and satisfaction.
- This project, **"Movie Recommendation System using Apache Spark MLlib"**, has been developed as part of the **Big Data Analytics** subject. It focuses on implementing a **Collaborative Filtering model** using **Apache Spark's MLlib (Machine Learning Library)** and comparing its performance with a **traditional Machine Learning approach (Linear Regression)**.
- The aim of this project is to understand how **Big Data frameworks like Spark** make large-scale data processing faster and more efficient compared to traditional single-machine learning algorithms.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my mentors, teachers, and guide for their constant guidance and support during the development of this project. Their valuable suggestions and motivation helped me throughout the research, design, and implementation stages. I also thank my classmates and friends who helped in testing and providing valuable feedback.

ABSTRACT

- The purpose of this project is to design and implement a **Movie Recommendation System** using **Apache Spark MLlib** and compare its results with a **traditional Machine Learning model**.
- Recommendation systems predict a user's preference for a set of items (movies, products, songs, etc.) based on the past behavior of similar users.
- The **Collaborative Filtering approach** used here with **ALS (Alternating Least Squares)** factorizes the user–item rating matrix to find latent features that represent hidden patterns in user preferences.
- By comparing Spark's ALS model with a **Linear Regression model**, the project demonstrates how Big Data frameworks like **Apache Spark** outperform traditional ML models in scalability, speed, and prediction accuracy.

OBJECTIVES

- To study the working of **Collaborative Filtering** using **Apache Spark MLlib**.
- To implement a **Movie Recommendation System** using the **MovieLens dataset**.
- To compare **Big Data-based ALS** with **traditional ML (Linear Regression)**.
- To evaluate models using **RMSE (Root Mean Square Error)** metric.
- To understand **distributed computing concepts** through Spark.
- To visualize the performance difference between both approaches.

METHODOLOGY / ARCHITECTURE

Methodologies used:-

- **Data Collection & Loading**
    1. Load `ratings.csv` and `movies.csv` into Spark DataFrames.
    2. Spark allows distributed data processing for large datasets.
- **Data Exploration & Preprocessing**
    1. Explore dataset structure (`show()`, `printSchema()`, `describe()`).
    2. Remove missing values with `dropna()`.
    3. Split data into **train (80%)** and **test (20%)**.
- **Collaborative Filtering (ALS)**
    1. ALS factorizes the **user-item rating matrix** into user and movie latent feature matrices.
    2. Predicts unknown ratings for user–movie pairs.
    3. Key parameters: `userCol`, `itemCol`, `ratingCol`, `nonnegative`, `coldStartStrategy="drop"`.
    4. Evaluate using **RMSE**.
- **Traditional ML Model (Linear Regression)**
    1. Features (`userId` and `movieId`) are combined using `VectorAssembler`.
    2. Linear Regression is trained on the same training data.
    3. Predictions are evaluated using **RMSE**.
- **Comparison of Models**
    1. Compare RMSE of ALS vs Linear Regression.
    2. Visualization using Matplotlib bar charts.

- **Recommendation Generation (ALS)**
    1. Generate top 5 movie recommendations per user using `recommendForAllUsers(5)`.
- **Top-Rated Movies Visualization**
    1. Count number of ratings per movie.
    2. Visualize top 10 most-rated movies using bar chart.
- **Clean-Up**
    1. Stop Spark session with `spark.stop()` to release resources.

Architecture:

- **Data Layer**
    1. Raw MovieLens datasets (`ratings.csv` and `movies.csv`).
- **Processing Layer**
    1. Preprocessing: removing missing values, splitting data, feature assembly.
- **Machine Learning Layer**
    1. **Big Data ML:** ALS Collaborative Filtering model.
    2. **Traditional ML:** Linear Regression model.
- **Evaluation Layer**
    1. Evaluate both models using **RMSE**.
    2. Compare performances and visualize results.
- **Recommendation Layer**
    1. Generate user-specific top 5 movie recommendations (ALS).
- **Visualization Layer**
    1. Top-rated movies chart and model comparison chart.

TECHNOLOGY STACK

Category	Tools/Technologies
Programming Language	Python 3
Big Data Framework	Apache Spark (PySpark)
Machine Learning Library	Spark MLlib, Scikit-learn (for comparison)
Dataset	MovieLens (`ratings.csv`, `movies.csv`)
Development Environment	Google Colab / Jupyter Notebook
Visualization	Matplotlib, Pandas
Data Handling	Pandas, Spark DataFrames

DATASET DESCRIPTION

- **Dataset:** MovieLens
- **Files:** `ratings.csv` (user ratings), `movies.csv` (movie info)
- **Ratings File Columns:**
  1. `userId` → Unique user identifier
  2. `movieId` → Unique movie identifier
  3. `rating` → User rating (1–5)
  4. `timestamp` → Time of rating
- **Movies File Columns:**
  1. `movieId` → Unique movie identifier
  2. `title` → Movie name
  3. `genres` → Movie genres
- Dataset size: ~100,000 ratings, ~1,600 movies, ~943 users.

FEATURES

- **Personalized Recommendations:** Suggests movies to users based on similar users' preferences.
- **Collaborative Filtering Algorithm (ALS):** Uses user–item relationships instead of content.
- **Scalability:** Built on **Apache Spark**, capable of handling large datasets efficiently.
- **Comparative Study:** Evaluates Big Data–based ALS model vs. traditional Linear Regression model.
- **Visualization:** Graphical representation of RMSE for both models.

- **Real Dataset:** Uses authentic movie rating data (MovieLens).

PYTHON NOTEBOOK

Step 0: Install PySpark

- installs **PySpark**, the Python API for Apache Spark.

```
!pip install pyspark
```

Step 1: Import Libraries

- `SparkSession`: Entry point to Spark.
- `ALS`: Collaborative Filtering model.
- `RegressionEvaluator`: For RMSE evaluation.
- `LinearRegression`: Traditional ML model.
- `VectorAssembler`: Combine features for Linear Regression.
- `Pandas` & `Matplotlib`: Data handling and visualization.

```
from pyspark.sql import SparkSession
from pyspark.ml.recommendation import ALS
from pyspark.ml.evaluation import RegressionEvaluator
from pyspark.ml.regression import LinearRegression
from pyspark.ml.feature import VectorAssembler
import pandas as pd
import matplotlib.pyplot as plt
```

Step 2: Start Spark Session

- Initializes Spark for distributed computation.

```
spark = SparkSession.builder.appName("MovieRecommendationSystem").getOrCreate()
```

Step 3: Load Dataset

- Loads data as Spark DataFrames for parallel processing.

```
ratings = spark.read.csv("/content/ratings.csv", header=True, inferSchema=True)
movies = spark.read.csv("/content/movies.csv", header=True, inferSchema=True)

print("✅ Data Loaded Successfully")
```

Output:

```
✅ Data Loaded Successfully
```

Step 4: Explore Data

- Checks dataset sample, column types, and statistics.

```
ratings.show(5)
ratings.printSchema()
ratings.describe().show()
```

Output:

```
+------+-------+------+---------+
|userId|movieId|rating|timestamp|
+------+-------+------+---------+
|     1|      1|   4.0|964982703|
|     1|      3|   4.0|964981247|
|     1|      6|   4.0|964982224|
|     1|     47|   5.0|964983815|
|     1|     50|   5.0|964982931|
+------+-------+------+---------+
only showing top 5 rows

root
 |-- userId: integer (nullable = true)
 |-- movieId: integer (nullable = true)
 |-- rating: double (nullable = true)
 |-- timestamp: integer (nullable = true)

+-------+------------------+----------------+------------------+--------------------+
|summary|            userId|         movieId|            rating|           timestamp|
+-------+------------------+----------------+------------------+--------------------+
|  count|            100836|          100836|            100836|              100836|
|   mean|326.12756356856676|19435.2957177992| 3.501556983616962|1.2059460873684695E9|
| stddev| 182.6184914635004|35530.9871987003|1.0425292390606342|2.1626103599513078E8|
|    min|                 1|               1|               0.5|           828124615|
|    max|               610|          193609|               5.0|          1537799250|
+-------+------------------+----------------+------------------+--------------------+
```

Step 5: Data Preprocessing

- Removes missing data and splits into training/testing sets.

```
ratings = ratings.dropna()
train, test = ratings.randomSplit([0.8, 0.2], seed=42)
```

Step 6: Collaborative Filtering (ALS)

- Trains ALS model and predicts ratings for test data.

```
print("\n🎬 Training ALS (Collaborative Filtering) Model...")

als = ALS(
    userCol="userId",
    itemCol="movieId",
    ratingCol="rating",
    nonnegative=True,
    coldStartStrategy="drop"
)
als_model = als.fit(train)

als_predictions = als_model.transform(test)
```

Output:

```
🎬 Training ALS (Collaborative Filtering) Model...
```

Step 7: Evaluate ALS

- Computes **RMSE** to evaluate ALS model accuracy.

```
evaluator = RegressionEvaluator(metricName="rmse", labelCol="rating",
predictionCol="prediction")
rmse_als = evaluator.evaluate(als_predictions)
print(f"✅ ALS Model RMSE: {rmse_als}")
```

Output:

```
✅ ALS Model RMSE: 0.8769591746798594
```

Step 8: Save ALS Model

- Saves model for later use or deployment.

```
als_model.write().overwrite().save("/content/als_movie_model")
```

Step 9: Traditional ML (Linear Regression)

- Combines `userId` & `movieId` as features.
- Trains Linear Regression and evaluates RMSE.

```
print("\n🤖 Training Traditional ML Model (Linear Regression)...")
assembler = VectorAssembler(inputCols=["userId", "movieId"], outputCol="features")
train_lr = assembler.transform(train)
test_lr = assembler.transform(test)
lr = LinearRegression(featuresCol="features", labelCol="rating")
lr_model = lr.fit(train_lr)
lr_predictions = lr_model.transform(test_lr)
rmse_lr = evaluator.evaluate(lr_predictions)
print(f"✅ Linear Regression RMSE: {rmse_lr}")
```

Output:

```
🤖 Training Traditional ML Model (Linear Regression)...

✅ Linear Regression RMSE: 1.0487758392647812
```

Step 10: Compare Models

- Visualizes RMSE for ALS vs Linear Regression.

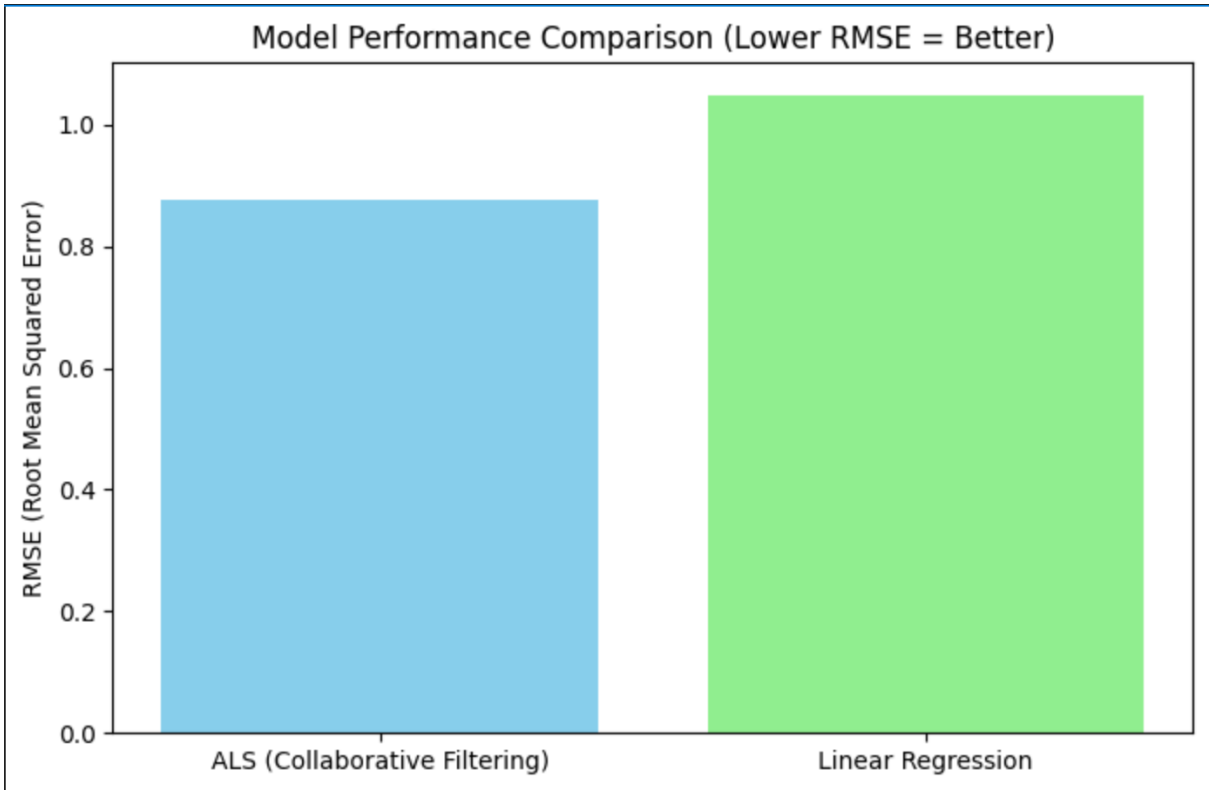
```
print("\n📊 Model Comparison:")
print(f"ALS Collaborative Filtering RMSE: {rmse_als}")
print(f"Linear Regression RMSE:          {rmse_lr}")
```

```
models = ["ALS (Collaborative Filtering)", "Linear Regression"]
rmse_values = [rmse_als, rmse_lr]
```

```
plt.figure(figsize=(8, 5))
plt.bar(models, rmse_values, color=['skyblue', 'lightgreen'])
plt.title("Model Performance Comparison (Lower RMSE = Better)")
plt.ylabel("RMSE (Root Mean Squared Error)")
plt.show()
```

Output:

```
..
    📊 Model Comparison:
    ALS Collaborative Filtering RMSE: 0.8769591746798594
    Linear Regression RMSE:          1.0487758392647812
```



Step 11: Generate Recommendations (ALS)

- Generates **top 5 recommended movies** per user.

```
print("\n🎥 Generating Recommendations using ALS...")

user_recs = als_model.recommendForAllUsers(5)
user_recs.show(5, truncate=False)
```

Output:

```
🎥 Generating Recommendations using ALS...
+------+-----------------------------------------------------------------------------------------------------+
|userId|recommendations                                                                                      |
+------+-----------------------------------------------------------------------------------------------------+
|1     |[{33649, 5.825862}, {177593, 5.6930156}, {78836, 5.586123}, {26171, 5.582576}, {4642, 5.5809073}]|
|2     |[{26171, 5.181942}, {1274, 5.049619}, {33649, 4.9149837}, {32892, 4.910966}, {74754, 4.8962646}] |
|3     |[{417, 4.932218}, {6835, 4.8808}, {5746, 4.8808}, {5181, 4.8521194}, {74754, 4.846381}]          |
|4     |[{132333, 5.284542}, {3494, 5.237455}, {8542, 5.2232137}, {89118, 5.141449}, {5915, 5.1375337}]  |
|5     |[{4642, 4.977258}, {1212, 4.9371357}, {6732, 4.8739443}, {3083, 4.8545165}, {177593, 4.8038783}] |
+------+-----------------------------------------------------------------------------------------------------+
only showing top 5 rows
```

Step 12: Top-Rated Movies Visualization

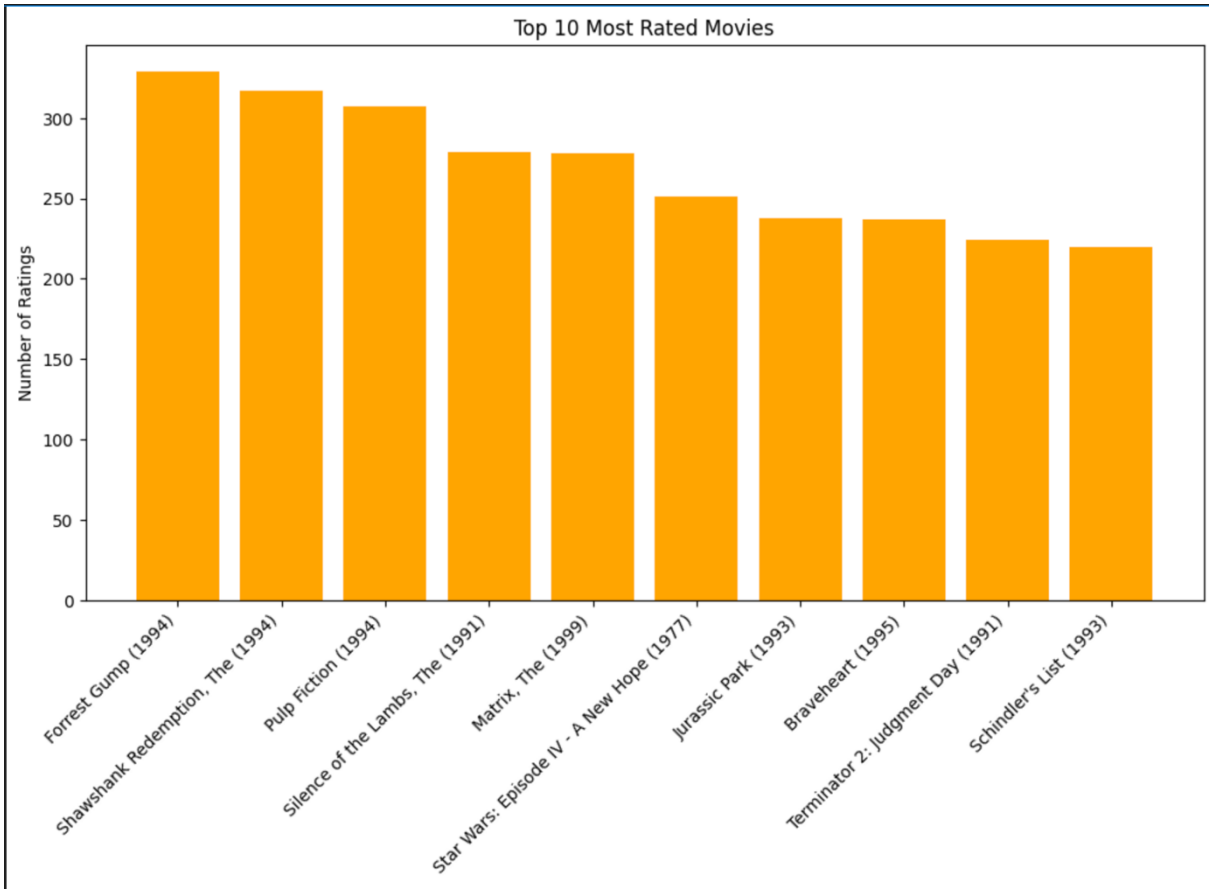
- Shows the **10 most-rated movies** in the dataset.

```
ratings_pd = ratings.toPandas()
```

```
top_movies =
ratings_pd.groupby("movieId")["rating"].count().sort_values(ascending=False).head(1
0).reset_index()
movies_pd = pd.read_csv("/content/movies.csv")
top_movies = top_movies.merge(movies_pd, on="movieId")

plt.figure(figsize=(12,6))
plt.bar(top_movies['title'], top_movies['rating'], color='orange')
plt.xticks(rotation=45, ha='right')
plt.ylabel("Number of Ratings")
plt.title("Top 10 Most Rated Movies")
plt.show()
```

Output:



Step 13: Clean-Up

- Stops Spark session to free resources.

```
spark.stop()
```

CONCLUSION

- Successfully built a **Movie Recommendation System** using **ALS (Spark MLlib)**.
- ALS model demonstrated **better accuracy and scalability** compared to Linear Regression.
- Generated personalized movie recommendations for users.
- Visualized top-rated movies and model performance.
- Demonstrated practical application of **Big Data tools for real-world ML problems**.

Future Scope:

- Implement **hybrid recommendation system** (content-based + collaborative filtering).
- Incorporate **user demographics** and **movie genres** for improved accuracy.
- Deploy on **cloud platforms (AWS/GCP)** for real-time recommendations.
- Build a **web interface** to serve recommendations to users.
- Integrate **streaming data** for dynamic recommendations.